



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Compiladores

Analizador sintáctico descendente recursivo

Práctica 3

Presenta:

Ramírez Juárez María Fernanda - 321204747

Rojo Peña Manuel Ianluck - 118005762

Profesor:

Pablo Martínez Yessica Janeth

Ayudante de laboratorio:

Salazar González Pedro Yamil

Grupo: 7010, 2027-1

Fecha de entrega:

21 de septiembre, 2026

Objetivo

En esta práctica nuestro objetivo fue implementar un analizador sintáctico descendente recursivo para MINIC, capaz de procesar los tokens que produce el Lexer de la Práctica 2 y determinar si el programa puede derivarse a partir de la gramática establecida del lenguaje.

Este Parser ocupa el lugar intermedio entre el Lexer y la futura construcción del AST, en esta entrega solo validamos la estructura, por lo que no construimos árboles de sintaxis, tabla de símbolos ni validaciones de tipos; sin embargo, cada función del parser corresponde ya a un no terminal de la gramática, de modo que en la siguiente práctica bastará con que cada una devuelva el nodo que reconoce, sin cambiar el recorrido.

Concretamente, el objetivo es:

1. Convertir el lexer en un productor incremental de tokens, sin impresión como efecto secundario
2. Mantener un token de anticipación para decidir qué producción aplicar
3. Reconocer declaraciones, asignaciones, impresión, condicionales, ciclos y bloques
4. Reconocer expresiones respetando la precedencia y asociatividad publicadas
5. Reportar errores léxicos y sintácticos con su posición en `stderr`
6. Recuperarnos de los errores para seguir analizando el resto del archivo
7. Verificar que toda la entrada se consuma hasta `TOKEN_EOF`
8. Informar el resultado general en `stdout` con un código de salida adecuado

Evolución respecto de la Práctica 2

Módulos modificados o agregados

- `lexer.h` y `lexer.c`: se reemplazó `lexer_scan` por una interfaz incremental que entrega un token por llamada, y se agregó la `enum LexerStatus`.
- `main.c`: ahora coordina la inicialización del lexer y del parser, el análisis, la liberación de recursos y el código de salida.
- `parser.h`, `parser_internal.h` y `parser.c`: módulos nuevos con el estado del parser y las operaciones sobre el flujo de tokens.
- `grammar.h` y `grammar.c`: módulo nuevo con una función por cada no terminal de la gramática.
- `token`, `keywords` y `buffer_manage` se conservan sin cambios.

Interfaz incremental incorporada al lexer

En la Práctica 2, nuestro `lexer_scan` en un ciclo que recorría el archivo completo e imprimía cada token en cuanto lo reconocía. Lo tuvimos que cambiar para el parser, ahora el lexer entrega *un* token cada vez que se le pide y conserve su estado (archivo, posición y caracter precargado) para la siguiente llamada. Por ello la interfaz pública quedó:

- `int lexer_init(Lexer *lexer, FILE *source)`: antes era una función `static` que usaba `lexer_scan` internamente, ahora es pública, recibe directamente el archivo a procesar, valida sus argumentos y precarga el primer caracter.
- `LexerStatus lexer_next_token(Lexer *lexer, Token *out)`: sustituye a `lexer_scan`. Conserva exactamente las mismas ramas de clasificación, pero cada una termina con un `return` en cuanto se forma el primer token, el ciclo interno solo se repite para saltar espacios y comentarios. Al llegar al final devuelve `TOKEN_EOF`, y las llamadas posteriores lo vuelven a devolver.
- `void lexer_destroy(Lexer *lexer)`: deja el lexer en un estado seguro, no libera memoria porque el lexer no reserva ninguna ni es dueño del archivo.
- `int lexer_print_tokens(FILE *source)`: conserva el comportamiento de las prácticas 1 y 2, pero construido sobre la nueva interfaz, solo se usa con la opción `-t` de `main.c`.

Para que el que pida un token pueda distinguir entre un token válido y un fallo del propio Lexer, agregamos `LexerStatus`:

```
1 /* Resultado de pedir un token */
2 typedef enum {
3     LEXER_STATUS_OK ,
4     LEXER_STATUS_IO_ERROR ,
5     LEXER_STATUS_MEMORY_ERROR
6 } LexerStatus;
```

Código 1: Enumeración `LexerStatus`, definida en `lexer.h`.

Así quedan separadas dos clases de fallo, un caracter inválido del programa fuente sigue siendo un token `ERROR` entregado con `LEXER_STATUS_OK`, mientras que un error de lectura o de memoria impide obtener el token y se informa con el estado correspondiente.

Eliminación de la impresión obligatoria de tokens

- `make_token`: sustituye a `emit_token`, antes construíamos, imprimíamos y destruíamos el token en la misma función, ahora `make_token` solo lo construye en `out` y devuelve `LEXER_STATUS_MEMORY_ERROR` si `token_init` falla.
- `scan_keyword` y `scan_integer`: ahora reciben el `Token *out` donde deben dejar el resultado y devuelven un `LexerStatus`. Ante un fallo de memoria regresan `LEXER_STATUS_MEMORY_ERROR` sin imprimir nada, el mensaje lo emite quien consume el token.

De esta forma, solicitar un token nunca escribe en `stdout` ni en `stderr`, la única función que imprime tokens es `lexer_print_tokens`.

Refactorizaciones realizadas y su justificación

Además de lo anterior, corregimos `skip_comment_content`. En la Práctica 2 la función solo se detenía en `\n` y además lo consumía:

```

1  /* Practica 2: descartamos el contenido de un comentario hasta \n o EOF */
2  static void
3  skip_comment_content(Lexer *lex)
4  {
5      while (lexer_check(lex) != EOF && lexer_check(lex) != '\n')
6          lexer_advance(lex);
7      if (lexer_check(lex) == '\n')
8          lexer_advance(lex);
9  }
10
11 /* Practica 3: descartamos el contenido de un comentario hasta \n, \r o EOF */
12 static void
13 skip_comment_content(Lexer *lex)
14 {
15     while (lexer_check(lex) != EOF && lexer_check(lex) != '\n' && lexer_check(
lex) != '\r')
16         lexer_advance(lex);
17 }
```

Código 2: `skip_comment_content` en la Práctica 2 y en la Práctica 3.

Con finales de línea que usan solo `'\r'`, el comentario se tragaba las líneas siguientes. Ahora el comentario termina en cualquiera de los dos caracteres, y el salto de línea se deja para el ciclo de `lexer_next_token`, que ya lo ignora como espacio en blanco.

Otros cambios menores en `lexer_next_token`: la verificación de `fehler` se movió a la rama de fin de archivo, y agregamos la guarda `c != '\0'` antes de `strchr`, ya que `strchr` encuentra el terminador de la cadena y habría clasificado un byte nulo como operador.

En `main.c` ahora:

- Validamos que se reciba exactamente un archivo, mostrando `Uso: ./minic <archivo.mc>` en caso contrario, y que el archivo pueda abrirse.
- `parse_file` inicializa el lexer y el parser, analiza el programa, libera ambos y traduce el resultado a un código de salida: 0 si el programa es correcto, 1 si tiene errores léxicos o sintácticos, y 2 para errores de uso, apertura, lectura o memoria.
- Conservamos la opción `-t` para imprimir tokens, únicamente como regresión de las prácticas anteriores.

Funcionalidades léxicas conservadas

Se conservan la lectura del archivo, el sistema de tokens, el almacenamiento de lexemas, el seguimiento de línea y columna (incluyendo `\r\n`), el reconocimiento de identificadores, palabras reservadas, literales enteros y booleanos, operadores simples y compuestos y delimitadores, el manejo de espacios y comentarios, la detección de errores léxicos, la generación de `TOKEN_EOF` y la

administración de memoria de `buffer_manage`. Las pruebas tanto públicas (del ayudante) como las propias de las Prácticas 1 y 2 siguen pasando en modo `-t`.

Arquitectura e interfaz

Organización del módulo parser

Dividimos el parser en dos niveles, cada uno con su propio archivo fuente:

- `parser.c`: el *flujo de tokens* pide tokens al Lexer, mantiene el token de anticipación, consume, reporta errores y sincroniza. No sabe nada de la gramática.
- `grammar.c`: la *gramática* una función por no terminal, que solo usa las operaciones de `parser.c` y nunca toca directamente al lexer.

Y tres encabezados, según quién necesita ver qué:

- `parser.h`: la interfaz pública, lo único que ve `main.c`.
- `parser_internal.h`: declara las operaciones auxiliares de `parser.c` que usa `grammar.c`. No pueden ser `static`, porque viven en otro archivo, pero tampoco deben estar en `parser.h`, porque `main.c` no tiene por qué avanzar tokens o sincronizar.
- `grammar.h`: declara únicamente `parse_program`, el punto de entrada de la gramática, el resto de las funciones de `grammar.c` son `static`.

Esta separación también la pensamos para la siguiente práctica, cuando cada función de la gramática construya nodos del AST, `grammar.c` crecerá considerablemente, mientras que el manejo de tokens de `parser.c` no tendrá que modificarse.

La base inicial proporcionada exponía en `parser.h` las operaciones `parser_advance`, `parser_check`, `parser_match` y `parser_current`, nosotros en cambio redujimos la interfaz pública a tres funciones:

```
1 int parser_init(Parser *parser, Lexer *lexer);
2 /* Analiza el programa completo: program ::= statement_list TOKEN_EOF */
3 ParseResult parser_parse_program(Parser *parser);
4 /* Libera los tokens que conserve el parser, exactamente una vez */
5 void parser_destroy(Parser *parser);
```

Código 3: Interfaz pública de `parser.h`.

El motivo es que `main.c` solo necesita inicializar, analizar y liberar, las demás operaciones son detalles internos del recorrido, por lo que pasaron a `parser_internal.h`. `parser_current` dejó de ser necesaria, porque la gramática consulta el *lookahead* con `parser_check`, y agregamos `parser_parse_program`, la operación que inicia el análisis completo que la base dejaba pendiente.

Representación del estado y token de anticipación

```

1  /* Resultado del analisis completo de un programa */
2  typedef enum {
3      PARSE_OK,                      /* programa sintacticamente correcto */
4      PARSE_INVALID,                 /* hubo errores lexicos o sintacticos */
5      PARSE_INTERNAL_FAILURE /* fallo de E/S o de memoria al pedir tokens */
6  } ParseResult;
7
8  typedef struct {
9      Lexer *lexer;
10     Token current;                 /* token de anticipacion */
11     Token previous;                /* ultimo token consumido */
12     int has_current;
13     int has_previous;
14     int had_error;                 /* hubo algun error lexico o sintactico */
15     int panic_mode;                /* suprime errores en cascada hasta sincronizar */
16     int internal_failure; /* el lexer fallo; el analisis se detiene */
17 } Parser;

```

Código 4: Estructuras ParseResult y Parser, definidas en parser.h.

Brevemente, `current` es el token de anticipación con el que decidimos qué producción aplicar, y `previous` conserva el último token consumido; `has_current` y `has_previous` indican si cada campo contiene un token válido que deba liberarse. `had_error` registra si hubo cualquier error léxico o sintáctico, `panic_mode` controla la recuperación e `internal_failure` detiene el análisis cuando el lexer falla. Finalmente con `ParseResult` traducimos este estado final a uno de tres resultados.

- Con `LEXER_STATUS_OK`, `lexer_next_token` entrega un token cuyo lexema fue reservado por `token_init`; quien lo recibe adquiere su propiedad.
- Con cualquier otro estado, `out` no es válido y no debe destruirse.
- El parser es dueño de `current` y `previous` mientras su bandera esté activa. Al avanzar, destruye el `previous` anterior y *mueve* `current` a `previous`, el apuntador al lexema cambia de campo, sin copiarse ni liberarse.
- Los tokens `ERROR` se destruyen en cuanto se reportan, sin llegar a `current`.
- `parser_destroy` libera ambos tokens exactamente una vez, incluso si hubo errores o fallos.
- El `Lexer` no reserva memoria propia, y ni este ni el `Parser` son dueños del archivo, que cierra `main.c`.

Cuando `lexer_next_token` devuelve un estado distinto de `LEXER_STATUS_OK`, el parser imprime el mensaje correspondiente, activa `internal_failure` y deja `has_current` en cero. A partir de ahí, `parser_end` devuelve verdadero, las funciones de error dejan de reportar y todas las funciones de la gramática se retiran, por lo que el análisis termina sin leer tokens inválidos. `parser_parse_program` devuelve entonces `PARSE_INTERNAL_FAILURE` y `main.c` termina con código 2.

Funciones de parser.c

```

1  /* Preparamos el parser, precargando el primer token de anticipacion */
2  int
3  parser_init(Parser *parser,
4             Lexer *lexer)
5  {
6      if (parser == NULL)
7          return 0;
8
9      memset(parser, 0, sizeof(*parser));
10
11     if (lexer == NULL)
12         return 0;
13
14     parser->lexer = lexer;
15     return fetch_token(parser);
16 }
17
18 /* Analizamos el programa completo y traducimos el estado final a un
19    ParseResult */
20 ParseResult
21 parser_parse_program(Parser *parser)
22 {
23     if (parser == NULL || parser->lexer == NULL || !parser->has_current)
24         return PARSE_INTERNAL_FAILURE;
25
26     parse_program(parser);
27
28     if (parser->internal_failure)
29         return PARSE_INTERNAL_FAILURE;
30     return parser->had_error ? PARSE_INVALID : PARSE_OK;
31 }
32
33 /* Liberamos los tokens que conserva el parser; el lexer no le pertenece */
34 void
35 parser_destroy(Parser *parser)
36 {
37     if (parser == NULL)
38         return;
39
40     if (parser->has_current) {
41         token_destroy(&parser->current);
42         parser->has_current = 0;
43     }
44     if (parser->has_previous) {
45         token_destroy(&parser->previous);
46         parser->has_previous = 0;
47     }
48     parser->lexer = NULL;
49 }

```

Código 5: Funciones públicas `parser_init`, `parser_parse_program` y `parser_destroy`.

`parser_init` pone todos los campos en cero *antes* de pedir el primer token, para que `parser_destroy` sea seguro aunque la inicialización falle. `parser_parse_program` es la operación pública que in-

voca `main.c`, el `main` llama a `parse_program`, el no terminal de la gramática, y traduce el estado final del parser a un `ParseResult`.

```

1  /* Consumimos el token actual, moviendolo a previous, y precargamos el
   siguiente */
2  int
3  parser_advance(Parser *parser)
4  {
5      if (parser == NULL
6          || parser->lexer == NULL
7          || parser->internal_failure)
8          return 0;
9
10     if (parser->has_current && parser->current.type == TOKEN_EOF)
11         return 1;
12
13     if (parser->has_previous) {
14         token_destroy(&parser->previous);
15         parser->has_previous = 0;
16     }
17
18     if (parser->has_current) {
19         parser->previous = parser->current;
20         parser->has_previous = 1;
21         parser->has_current = 0;
22     }
23
24     return fetch_token(parser);
25 }
26
27 /* Checamos si el token actual es del tipo indicado, sin consumirlo */
28 int
29 parser_check(const Parser *parser,
30             TokenType type)
31 {
32     return parser != NULL
33         && parser->has_current
34         && parser->current.type == type;
35 }
36
37 /* Consumimos el token actual solo si es del tipo indicado */
38 int
39 parser_match(Parser *parser,
40             TokenType type)
41 {
42     if (!parser_check(parser, type))
43         return 0;
44     parser_advance(parser);
45     return 1;
46 }

```

Código 6: Operaciones `parser_advance`, `parser_check` y `parser_match`.

`parser_advance` implementa el convenio de propiedad descrito arriba y nunca avanza más allá de `TOKEN_EOF`, de modo que ningún ciclo puede leer más allá del final. `parser_check` consulta el *lookahead* sin consumirlo, y `parser_match` lo consume solo si coincide, lo que usamos para las

partes opcionales de la gramática (la inicialización de una declaración, el `else` y los operadores binarios).

Las funciones restantes de `parser.c` son:

- `report_lexer_failure(...)`: imprime en `stderr` el mensaje de un fallo interno del Lexer, de memoria o de lectura.
- `fetch_token(...)`: pide tokens al lexer hasta obtener uno que no sea `ERROR` y lo deja en `current`. Cada `ERROR` se reporta, se destruye y marca el programa como inválido, ante un fallo interno activa `internal_failure`.
- `parser_previous(...)`: devuelve el último token consumido, o `NULL` si aún no hay ninguno.
- `parser_end(...)`: indica si ya no queda nada por analizar, ya sea porque el *lookahead* es `TOKEN_EOF` o porque hubo un fallo interno.
- `parser_current_error(...)`: atajo de `parser_error` sobre el token actual.
- `parser_consume(...)`: exige un token del tipo esperado, si está, lo consume y devuelve 1, si no, reporta el error sin fingir que el token existía, activa `panic_mode` y devuelve 0.

`parser_error` y `parser_sync` se describen en la sección de Diagnósticos y Recuperación.

Descenso recursivo y gramática

En un Parser descendente recursivo, cada no terminal se implementa como una función que inspecciona el *lookahead*, decide qué producción aplicar, consume los terminales esperados e invoca las funciones de los no terminales que aparecen en esa producción. Las producciones de sentencias que implementamos son:

```

program      ::= statement_list TOKEN_EOF ;
statement_list ::= statement* ;
statement    ::= declaration | assignment | print_statement
               | if_statement | while_statement | block ;
declaration  ::= type IDENTIFIER ( ASSIGN expression )? SEMICOLON ;
assignment   ::= IDENTIFIER ASSIGN expression SEMICOLON ;
print_statement ::= PRINT LPAREN expression RPAREN SEMICOLON ;
if_statement  ::= IF LPAREN expression RPAREN statement ( ELSE statement )? ;
while_statement ::= WHILE LPAREN expression RPAREN statement ;
block        ::= LBRACE statement* RBRACE ;

```

Las repeticiones `*` se traducen en ciclos `while`, las partes opcionales `?` en un `parser_match`, y cada terminal obligatorio en un `parser_consume`. Al inicio de `grammar.c` declaramos los prototipos de todas las funciones, porque la gramática es mutuamente recursiva, `parse_statement` llama a `parse_if_statement`, que vuelve a llamar a `parse_statement` para su cuerpo, así que no existe un orden de definición en el que todas aparezcan antes de usarse.

```

1  /* Reconocemos el programa completo y verificamos que termine en TOKEN_EOF */
2  void
3  parse_program(Parser *parser)
4  {
5      parse_statement_list(parser);
6      if (!parser->internal_failure && !parser_check(parser, TOKEN_EOF))
7          parser_current_error(parser, "el fin del archivo");
8  }

```

Código 7: Función `parse_program`, único punto de entrada de `grammar.c`.

Las funciones de sentencias son:

- `parse_statement_list`: llama a `parse_statement` mientras `parser_end` sea falso, por lo que un archivo vacío es válido.
- `parse_statement`: elige la producción con un `switch` sobre el tipo del lookahead. `INT` o `BOOL` inicia una declaración, `IDENTIFIER` una asignación, `PRINT` una impresión, `IF` un condicional, `WHILE` un ciclo y `LBRACE` un bloque. Cualquier otro token se reporta como se esperaba una sentencia y se consume. Al final, si quedó activo `panic_mode`, llama a `parser_sync`.
- `parse_declaration`: consume el tipo, exige un identificador, acepta opcionalmente `= expresión` y exige `;`.
- `parse_assignment`: consume el identificador y exige `=`, una expresión y `;`.
- `parse_print_statement`: consume `print` y exige `(`, una expresión, `)` y `;`.
- `parse_if_statement`: consume `if`, exige la condición entre paréntesis, analiza el cuerpo con `parse_statement` y, si el lookahead es `else`, analiza la rama alternativa. Como el `if` más interno es el primero en ver el `else`, este se asocia siempre con el `if` más cercano que aún no tiene rama `else`.
- `parse_while_statement`: consume `while`, exige la condición entre paréntesis y analiza el cuerpo con `parse_statement`.
- `parse_block`: consume `{`, analiza sentencias hasta encontrar `}` o el fin del archivo, y exige el `}` de cierre.

En todas ellas, el primer token ya fue verificado por `parse_statement`, por eso se consume con `parser_advance` en lugar de `parser_consume`. Si algún `parser_consume` falla o una expresión deja activo `panic_mode`, la función se retira de inmediato y la recuperación queda a cargo de `parse_statement`.

Expresiones

Implementamos una función por cada nivel de precedencia, desde el de menor precedencia hasta el de mayor, la precedencia sale del anidamiento donde cada nivel obtiene sus operandos llamando al nivel inmediatamente superior, así que los operadores de mayor precedencia se reconocen completos

antes de volver a los de menor. Por ejemplo, en $5 + 2 * 3$, `parse_additive_expression` pide su segundo operando a `parse_multiplicative_expression`, que reconoce $2 * 3$ entero; así la agrupación es $5 + (2 * 3)$.

- `parse_expression`: punto de entrada que delega en el nivel de menor precedencia.
- `parse_or_expression`: operandos de `parse_and_expression` separados por `||`.
- `parse_and_expression`: operandos de `parse_equality_expression` separados por `&&`.
- `parse_equality_expression`: operandos de `parse_comparison_expression` separados por `==` o `!=`.
- `parse_comparison_expression`: operandos de `parse_additive_expression` separados por `<`, `<=`, `>` o `>=`.
- `parse_additive_expression`: operandos de `parse_multiplicative_expression` separados por `+` o `-`.
- `parse_multiplicative_expression`: operandos de `parse_unary_expression` separados por `*` o `/`.
- `parse_unary_expression`: si el *lookahead* es `-`, lo consume y se llama a sí misma, si no, delega en `parse_primary`.
- `parse_primary`: acepta un entero, `true`, `false`, un identificador o una expresión entre paréntesis, con cualquier otro token reporta `se esperaba una expresion`.

Los niveles binarios siguen el patrón *operando, luego while* (`parser_match(operador)`) *otro operando*. Ese ciclo reconoce la secuencia de izquierda a derecha, que corresponde a la asociatividad por la izquierda publicada ($a - b - c$ se agrupa como $(a - b) - c$). El operador unario, en cambio, se implementó con recursión hacia la derecha, en `- - x`, el primer `-` se aplica al resultado de reconocer `- x`, lo que corresponde a su asociatividad por la derecha. Los paréntesis alteran la precedencia porque `parse_primary` vuelve a `parse_expression`, reiniciando el recorrido desde el nivel más bajo dentro de ellos.

Los ciclos binarios también verifican `!parser->panic_mode`, para dejar de buscar operadores en cuanto un operando falla. Como en esta práctica no construimos AST, la agrupación no es observable en la salida; queda garantizada por la estructura de las funciones y se reflejará en los nodos de la siguiente práctica.

Diagnósticos y recuperación

Cuándo se emite un error sintáctico

Emitimos un error sintáctico cuando el *lookahead* no coincide con lo que exige la gramática, en cuatro lugares: cuando falla un `parser_consume`, cuando `parse_primary` no encuentra el inicio de una expresión; cuando `parse_statement` encuentra un token que no inicia ninguna sentencia;

y cuando `parse_program` termina sin estar en `TOKEN_EOF`.

Todos pasan por `parser_error`, que es nuestra implementación del `syntax_error` de la especificación:

```

1  /* Reportamos un error sintactico sobre un token y activamos panic_mode para
   *   evitar cascadas */
2  void
3  parser_error(Parser *parser,
4               const Token *token,
5               const char *what)
6  {
7      if (parser == NULL
8          || token == NULL
9          || parser->internal_failure)
10         return;
11
12     parser->had_error = 1;
13
14     if (parser->panic_mode)
15         return;
16     parser->panic_mode = 1;
17
18     if (token->type == TOKEN_EOF)
19         fprintf(stderr,
20                 "Error sintactico [%zu:%zu]: se esperaba %s, pero se encontro
21                 TOKEN_EOF.\n",
22                 token->line, token->column, what);
23     else
24         fprintf(stderr,
25                 "Error sintactico [%zu:%zu]: se esperaba %s, pero se encontro
26                 '%s'.\n",
27                 token->line, token->column, what, token->lexeme);
28 }

```

Código 8: Función `parser_error`.

Al concentrar el formato en una sola función, todos los diagnósticos respetan la estructura publicada y cada producción solo aporta la descripción de lo esperado, un token concreto entre comillas (‘;’, ‘)’) o una categoría con artículo (*una expresion, un identificador*).

Cuando el token encontrado es `TOKEN_EOF`, se imprime ese nombre en lugar de un lexema vacío. Si `panic_mode` ya está activo, el error se registra en `had_error` pero no se imprime, con lo que evitamos reportar en cascada errores derivados del primero.

Diferencia con un token léxico `ERROR`

Los tokens `ERROR` nunca llegan a la gramática, pues `fetch_token` los intercepta, imprime `Error lexico [linea:columna]: caracter invalido '<lexema>'`, marca `had_error`, los destruye y pide el siguiente token. Así cada carácter inválido produce un único diagnóstico léxico, se consume para no quedar detenidos en él, y no genera además un error sintáctico. Como `had_error` queda activo, un programa con errores léxicos nunca se reporta como correcto.

Puntos de sincronización

```
1  /* Descartamos tokens hasta un punto seguro para continuar el analisis tras un
   error */
2  void
3  parser_sync(Parser *parser)
4  {
5      if (parser == NULL)
6          return;
7
8      parser->panic_mode = 0;
9
10     while (!parser->internal_failure
11            && parser->has_current) {
12         switch (parser->current.type) {
13             case SEMICOLON:
14                 parser_advance(parser);
15                 return;
16
17             case INT:
18             case BOOL:
19             case IF:
20             case WHILE:
21             case PRINT:
22             case IDENTIFIER:
23             case LBRACE:
24                 return;
25
26             case RBRACE:
27             case ELSE:
28             case TOKEN_EOF:
29                 return;
30
31             default:
32                 parser_advance(parser);
33                 break;
34         }
35     }
36 }
```

Código 9: Función `parser_sync`.

Tras un error, la función que lo detectó se retira y `parse_statement` llama a `parser_sync`, que desactiva `panic_mode` y descarta tokens hasta encontrar un punto seguro:

- `;` se consume, porque marca el fin de la sentencia errónea.
- `int`, `bool`, `if`, `while`, `print`, un identificador o `{` no se consumen, porque pueden iniciar la siguiente sentencia.
- `}`, `else` y `TOKEN_EOF` no se consumen, porque pertenecen a la estructura exterior, el bloque que debe cerrarse, el `if` que puede tomar su `else`, o el programa.

Un `else` que no puede asociarse con ningún `if` llega a `parse_statement` como *lookahead*, se reporta como se esperaba una sentencia y se consume.

Toda llamada a `parse_statement` consume al menos un token, cada caso del `switch` consume su token inicial, y el caso por defecto consume el token inesperado después de reportarlo. Como `parse_statement_list` y `parse_block` llaman a `parse_statement` mientras no están en su token de cierre, cada iteración avanza en la entrada y ningún ciclo puede quedar detenido en el mismo token. A esto se suman `panic_mode`, que impide reportar más de un error por sentencia, y `parser_advance`, que nunca avanza más allá de `TOKEN_EOF`. Por ejemplo, para

```
int contador = 3

while (contador > 0 {
print(contador)
}
```

se reportan los tres errores esperados (el `;` faltante antes de `while`, el `)` faltante antes de `{` y el `;` faltante antes de `}`) y el análisis continúa hasta el final del archivo.

Consumo completo del archivo

`parse_statement_list` solo termina cuando el *lookahead* es `TOKEN_EOF` o hubo un fallo interno, así que cualquier token sobrante, como un `)` o un `}` después de un programa válido, llega a `parse_statement` y se reporta como inesperado. Además, `parse_program` verifica explícitamente que el lookahead final sea `TOKEN_EOF`, solo si toda la entrada se consumió sin errores, `parser_parse_program` devuelve `PARSE_OK` y `main.c` imprime `Programa sintacticamente correcto`.

Resultados y pruebas

Como indica el enunciado, y como hemos echo en las anteriores prácticas, las pruebas públicas no sustituyen las pruebas diseñadas por el equipo, así que además de correr las proporcionadas diseñamos casos propios enfocados en demostrar que el parser se recupera ante errores léxicos y sintácticos y recorre el archivo completo.

Pero antes de describirlos, explicamos cómo adaptamos el `Makefile` para esta práctica.

Cambios en el Makefile

Respecto de la Práctica 2, el `Makefile` cambió en los siguientes puntos:

- `SOURCES` incluye ahora `parser.c` y `grammar.c`, y los encabezados de los que depende el ejecutable se obtienen con `$(wildcard include/lexer/*.h include/parser/*.h)`, en lugar de listarlos uno por uno.
- La plantilla `run_test_dir` recibe ahora cuatro parámetros: directorio de entradas, directorio de salidas esperadas, banderas extra para `minic` y el nombre de la suite (un grupo de casos de prueba que se ejecutan juntos). Las salidas reales de cada suite se guardan en su propia carpeta `build/<suite>`, para que casos con el mismo nombre en suites distintas no se sobrescriban.

- En la Práctica 2 la plantilla solo comparaba `stdout`, ahora compara además `stderr` contra `<nombre>.err` y el código de salida contra `<nombre>.code`, siempre que esos archivos existan. Esto era necesario porque en un programa inválido `stdout` está vacío, así que comparar solo `stdout` dejaría pasar cualquier diagnóstico incorrecto. Los casos del Lexer, que solo tienen `.out`, siguen funcionando igual.
- Como en esta práctica el modo normal ejecuta el parser, las pruebas de las Prácticas 1 y 2 se corren con la opción `-t`, que conserva la impresión de tokens.
- Reorganizamos las pruebas propias en `tests/unit_tests/lexer/`, con los casos de las Prácticas 1 y 2, y `tests/unit_tests/parser/`, con los casos de esta práctica.

Los comandos implementados para las pruebas son:

- `make test`: corre todas las pruebas (`test-cli`, `test-unit` y `test-args`).
- `make test-cli`: corre las pruebas públicas de las tres prácticas.
- `make test-p1` y `make test-p2`: pruebas públicas de las Prácticas 1 y 2, en modo `-t`.
- `make test-p3`: ejecuta el script público de la Práctica 3 y revisa su resumen.
- `make test-unit`: corre las pruebas propias del lexer y del parser.
- `make test-unit-lexer`: pruebas propias de las Prácticas 1 y 2, en modo `-t`.
- `make test-unit-parser`: pruebas propias del parser, comparando `stdout`, `stderr` y código de salida.
- `make test-args`: valida la línea de comandos (sin archivo, dos archivos, solo `-t`, archivo inexistente y un directorio que no puede leerse), exigiendo en cada caso código 2, `stdout` vacío y un mensaje en `stderr`, además del texto exacto del mensaje de uso.

Como las pruebas públicas de la Práctica 3 no siguen el formato de las anteriores, pues traen un único `programa_correcto.out` y su propio script, `run_public_tests.sh`, en lugar de reimplementar su convención, `test-p3` ejecuta directamente el script. Como su salida pasa por `tee` para guardarla en `build/public-p3.log`, el código de salida que ve `make` es el de `tee` y no el del script.

Pruebas públicas de la Práctica 3:

```
[code/] > make test-p3
==== public-p3 ====
[OK] p01_empty
[OK] p02_declarations
[OK] p03_assignment_print
[OK] p04_expression_precedence
[OK] p05_if_else
[OK] p06_dangling_else
[OK] p07_while_blocks
```

```
[OK] p08_complete_program
[OK] p09_missing_semicolon
[OK] p10_missing_parenthesis
[OK] p11_missing_brace
[OK] p12_incomplete_expression
[OK] p13_operator_without_operand
[OK] p14_unexpected_token
[OK] p17_trailing_content
[OK] p15_multiple_errors
[OK] p16_lexical_error
```

Aprobadas: 17

Fallidas: 0

Pruebas propias del parser

Del mismo modo aplicamos con un `sh` para probar estas pruebas propias del parser. Cada caso consta de un `.mc` en `tests/unit_tests/parser/inputs/` y de tres archivos en `tests/unit_tests/parser/expected/`: el `.out` vacío, porque un programa inválido no debe escribir nada en `stdout`; el `.code`, con el código de salida 1; y el `.err`, con los diagnósticos exactos esperados, incluyendo línea y columna. En todos los casos colocamos a propósito un error en la última línea del archivo, si ese diagnóstico aparece, queda demostrado que el parser recorrió el archivo completo a pesar de los errores anteriores.

`ut_operadores_sin_operando`: operadores binarios a los que les falta un operando, y un `-` unario sin operando. Cada línea inválida produce exactamente un error, y la línea válida intermedia (línea 8) no produce ninguno.

```
int x = 1 + ;
x = * 2;
x = 1 + * 2;
x = (3 - );
print(4 < );
bool b = true && || false;
x = - ;
x = 10 / 2 * 3;
print(x ==)
```

```
Error sintactico [1:12]: se esperaba una expresion, pero se encontro ';''.
Error sintactico [2:4]: se esperaba una expresion, pero se encontro '*'.
Error sintactico [3:8]: se esperaba una expresion, pero se encontro '*'.
Error sintactico [4:9]: se esperaba una expresion, pero se encontro ')'.
Error sintactico [5:10]: se esperaba una expresion, pero se encontro ')'.
Error sintactico [6:17]: se esperaba una expresion, pero se encontro '||'.
Error sintactico [7:6]: se esperaba una expresion, pero se encontro ';''.
Error sintactico [9:10]: se esperaba una expresion, pero se encontro ')'.

```

En la línea 9 también falta el `;`, pero no se reporta, porque `panic_mode` ya está activo por el error anterior de la misma sentencia; así evitamos diagnósticos en cascada.

`ut_parentesis_y_llaves`: paréntesis de más y de menos en expresiones, en un `if` y en un `while`, y un bloque sin cerrar al final. Incluye también varios niveles de anidamiento válidos (línea 1 y bloques dentro de bloques) que no producen errores.

```
print((((1 + 2))));
x = ((1 + 2) * 3;
y = (1 + 2)) * 3;
if ((x > 0) { print(x); }
while (x > 0)) x = x - 1;
{
    { int z = (x * (y - 1)); }
    print(z);
```

```
Error sintactico [2:16]: se esperaba ')', pero se encontro ';''.
Error sintactico [3:11]: se esperaba ';', pero se encontro ')'.
Error sintactico [4:12]: se esperaba ')', pero se encontro '{'.
Error sintactico [5:13]: se esperaba una sentencia, pero se encontro ')'.
Error sintactico [9:0]: se esperaba ')', pero se encontro TOKEN_EOF.
```

En la línea 4 la sincronización se detiene en `{` sin consumirlo, así que el bloque siguiente se analiza normalmente; en la línea 5 el `)` sobrante se reporta como token inesperado y la asignación que le sigue se reconoce sin errores.

`ut_lexicos_y_sintacticos`: errores léxicos y sintácticos intercalados, incluso en la misma línea. Cada caracter inválido produce un único diagnóstico léxico, y solo hay error sintáctico cuando, sin ese caracter, la estructura queda rota: en la línea 1 el programa es válido sin `@` y `$`, mientras que en la línea 2, sin `@`, quedan `x 1` seguidos.

```
@ int x = 10 $;
print(x @ 1);
if (x # > 0) print(x;
while (x > 0) { x = x - 1 & ; }
bool b = true ~
```

```
Error lexico [1:0]: caracter invalido '@'.
Error lexico [1:13]: caracter invalido '$'.
Error lexico [2:8]: caracter invalido '@'.
Error sintactico [2:10]: se esperaba ')', pero se encontro '1'.
Error lexico [3:6]: caracter invalido '#'.
Error sintactico [3:20]: se esperaba ')', pero se encontro ';''.
Error lexico [4:26]: caracter invalido '&'.
Error lexico [5:14]: caracter invalido '~'.
Error sintactico [6:0]: se esperaba ';', pero se encontro TOKEN_EOF.
```

`ut_recuperacion_estructuras`: errores dentro de un `if/else` anidado en un `while`, un `else` huérfano y un `else` sin sentencia al final del archivo. Después de cada error las llaves se siguen emparejando correctamente, por lo que no aparece ningún `}` falso, y la línea 9 se reconoce dentro del ciclo sin errores.

```

int contador = 3;
while (contador > 0) {
    if (contador == 2) {
        x = ;
        print(x)
    } else {
        int = 3;
    }
    contador = contador - 1;
}
else print(0);
if (contador) print(1); else

```

Error sintactico [4:8]: se esperaba una expresion, pero se encontro ';'.

Error sintactico [6:2]: se esperaba ';', pero se encontro '}'.

Error sintactico [7:8]: se esperaba un identificador, pero se encontro '='.

Error sintactico [11:0]: se esperaba una sentencia, pero se encontro 'else'.

Error sintactico [13:0]: se esperaba una sentencia, pero se encontro TOKEN_EOF.

Los cuatro casos pasan:

```

[code/] > make test-unit-parser
==== unit-parser ====
PASS: ut_lexicos_y_sintacticos
PASS: ut_operadores_sin_operando
PASS: ut_parentesis_y_llaves
PASS: ut_recuperacion_estructuras
----
Total: 4 Pass: 4 Fail: 0

```

Regresión del lexer

Para comprobar que la refactorización del lexer conservó su comportamiento, corrimos de nuevo las pruebas públicas de las Prácticas 1 y 2 y nuestros casos propios del lexer, todos en modo `-t`. Como la interfaz incremental entrega exactamente la misma secuencia de tokens que producía `lexer_scan`, todas siguen pasando:

```

[code/] > make test-p2
==== public-p2 ====
PASS: p01_identifiers
PASS: p02_reserved_words
PASS: p03_case_sensitive
PASS: p04_booleans
PASS: p05_compound_operators
PASS: p06_incomplete_operators
PASS: p07_comments
PASS: p08_comment_at_eof
PASS: p09_slash_and_comments

```

```
PASS: p10_longest_match
PASS: p11_error_recovery
PASS: p12_complete_lexer
PASS: p13_crlf
----
Total: 13 Pass: 13 Fail: 0

[code/] > make test-p1 test-unit-lexer
==== public-p1 ====
PASS: p10_mixed_basic
PASS: p1_empty
PASS: p2_whitespace
PASS: p3_single_symbols
PASS: p4_integer
PASS: p5_integer_expression
PASS: p6_multiple_lines
PASS: p7_tabs_spaces
PASS: p8_error_character
PASS: p9_negative_number
----
Total: 10 Pass: 10 Fail: 0
==== unit-lexer ====
PASS: ut_comment_literal_newline
PASS: ut_compound_ops_and_comments
PASS: ut_dense_no_spaces
PASS: ut_error_recovery_mixed
PASS: ut_slashes
----
Total: 5 Pass: 5 Fail: 0
```

Validación de la línea de comandos

Por último, `make test-args` comprueba que los errores de uso y de apertura se distingan de los errores del programa fuente: en los seis casos el programa termina con código 2, sin escribir en `stdout` y con un mensaje en `stderr`.

```
[code/] > make test-args
==== args ====
PASS: sin archivo
PASS: mensaje de uso
PASS: dos archivos
PASS: solo -t
PASS: archivo inexistente
PASS: directorio (no legible)
----
Total: 6 Pass: 6 Fail: 0
```

Problemas encontrados

Separación entre `parser.h` y `parser_internal.h`

Al dividir el parser en `parser.c` (flujo de tokens) y `grammar.c` (gramática), nos encontramos con que las operaciones auxiliares (`parser_advance`, `parser_check`, `parser_match`, `parser_consume`, `parser_sync`, entre otras) estaban declaradas como `static` en `parser.c`, como sugería la base inicial, así que `grammar.c` no podía llamarlas.

La primera solución que consideramos fue declararlas en `parser.h`, pero ese encabezado lo incluye `main.c`, lo que le habría dado acceso a operaciones que no le corresponden, como avanzar tokens o sincronizar, realmente no habría problema en hacerlo, sin embargo tratamos de que todo esté bien definido. También evaluamos juntar todo en un solo `parser.c`, lo que elimina el problema, pero a cambio de un archivo de más de quinientas líneas que mezcla dos niveles de abstracción.

Optamos por un tercer encabezado, `parser_internal.h`, que declara las operaciones auxiliares y que solo incluyen `parser.c` y `grammar.c`. Así `parser.h` quedó con la interfaz mínima que necesita `main.c` (`parser_init`, `parser_parse_program` y `parser_destroy`), y cada archivo fuente conserva una sola responsabilidad.

Desbordamiento de pila por anidamiento

Al probar entradas extremas encontramos que un archivo con alrededor de cien mil paréntesis anidados, por ejemplo `x = (((...1...)))`; , termina con `Segmentation fault`. La causa es propia del descenso recursivo, cada nivel de paréntesis recorre toda la cadena `parse_expression` → `parse_or_expression` → ... → `parse_primary`, y vuelve a empezar dentro de ellos, así que cada nivel agrega varias llamadas a la pila hasta desbordarla. Lo mismo ocurre con bloques, `if` o - unarios encadenados en cantidades similares.

Con niveles de anidamiento realistas (decenas o incluso cientos) el parser funciona correctamente, como muestra `ut_parenthesis_y_llaves`. Una solución sería agregar un contador de profundidad al parser y detener el análisis de forma controlada al superar un límite, decidimos no incluirla en esta entrega por tratarse de un caso artificial muy extremo, pero la dejamos registrada como limitación conocida.

Conclusiones

Esta práctica nos mostró como es la implementación de una gramática en C, donde la parte más directa de un parser descendente recursivo es la traducción de la gramática, cada producción se convierte en una función, cada repetición en un ciclo, cada parte opcional en un `match` y cada terminal obligatorio en un `consume`. Incluso la precedencia y la asociatividad de los operadores, que parecían requerir lógica especial, quedaron resueltas solo con el orden en que las funciones se llaman entre sí.

La recuperación ante errores nos obligó a pensar mejor en el diseño e implementación del parser, no bastaba con un `print` en cada error encontrado, había que garantizar que cada ruta consumiera algún token o devolviera el control a quien pudiera procesarlo, porque de lo contrario el parser se

quedaba detenido o repetía el mismo error indefinidamente. La interfaz incremental, por su parte, nos obligó a definir con precisión quién es dueño de cada token y cuándo se libera, algo que en la Práctica 2 no hacía falta porque cada token se imprimía y se destruía en el mismo lugar.

Supongamos y en cierto modo afirmamos que Esta organización debería facilitar la construcción del AST en la siguiente práctica pues cada función de `grammar.c` ya corresponde a un no terminal, así que bastará con que en lugar de `void` devuelva el nodo que reconocem, como `parse_declaration` un nodo de declaración, `parse_primary` una hoja con un literal o un identificador, y así sucesivamente. Como el manejo de tokens vive en `parser.c` y gracias a implementar la práctica de este modo, todos los futuros cambios e implementaciones que deberemos hacer para la futura práctica se concentrarán en `grammar.c`.

División del trabajo

Para esta práctica, debido a circunstancias sucitadas por ambos integrantes, se complicó la participación activa de ambos integrantes; por ello la entrega fue realizada demasiado tarde.

Sin embargo para compensar esta situación se entregará la práctica 4 mucho antes de la fecha de entrega establecida, y será la compañera Fernanda quien aporte más a su realización para compensar su escasez en la práctica 3.

La compañera *María Fernanda* realizó todas las adaptaciones y mejoras correspondientes al Lexer, además de aportar en discusiones sobre la implementación de código y redacción del reporte. El compañero *Manuel Ianluck* por su parte, realizó el código correspondiente al Parser y a la gramática.

Uso de herramientas de Inteligencia Artificial

Para la realización de esta práctica no se llevó a cabo ninguna utilización de inteligencia artificial de ningún tipo, más que para el texto del README.md. Se le pidió lo siguiente:

En base a este reporte `<practica.tex>`, este README realizado con anterioridad, genera el texto a agregar en los siguientes encabezados:

```
### Propiedad de tokens y lexemas, ## Funcionalidades implementadas,  
## Pruebas, ## Problemas conocidos, ## Notas de ejecución
```

para completar el README de mi Práctica 3. El texto debe ser muy breve pues todo lo implementado ya está detallado en el reporte, estas secciones del README funcionan como un resumen muy breve de lo implementado el código.

Se le pasó el texto del archivo LaTeX del reporte aquí presente y el texto del README echo en la Práctica 2, para que simplemente resuma lo necesario de nuestro reporte ya echo. Esto por la falta de tiempo que el equipo contó para refinar este archivo.

References

- [1] ppreference.com. (n.d.). C language. Disponible en <https://en.cppreference.com/c/language>
- [2] ppreference.com. (n.d.). C11. Disponible en <https://en.cppreference.com/c/11>
- [3] cppreference.com. (n.d.). File input/output, ferror. Disponible en <https://en.cppreference.com/c/io/ferror>
- [4] Free Software Foundation. (n.d.). GNU make. Disponible en <https://www.gnu.org/software/make/manual/make.html>